

1. Chức năng: Đăng yêu cầu trao đổi (Create Task)

A. Luồng chạy của Function (Workflow):

1. **Giao diện (View):** Người dùng nhập tiêu đề, mô tả, số giờ và deadline tại trang `create_request.html`. Khi nhấn "Đăng yêu cầu", JavaScript thu thập dữ liệu và gửi một POST Request đến đầu cuối `/api/tasks/`
2. **Điều hướng (Controller/Router):** Hàm `create_task` trong `backend/routers/task.py` tiếp nhận yêu cầu. Đầu tiên, nó thực hiện kiểm tra tính hợp lệ (Validation) thông qua Schema `TaskCreate`.
3. **Xử lý nghiệp vụ (Business Logic):**
 - a. Truy vấn đối tượng **User** từ Database để kiểm tra số dư (`balance_available`).
 - b. Nếu số dư đủ, thực hiện trừ `balance_available` và cộng vào `balance_locked` (Ký quỹ).
 - c. Khởi tạo một thực thể **Task** mới với trạng thái ban đầu là `open`.
4. **Lưu trữ (Model/Database):** Sử dụng SQLAlchemy để thực hiện `db.add()` và `db.commit()`. Nếu có lỗi xảy ra, hệ thống gọi `db.rollback()` để hoàn trả tiền cho người dùng.
5. **Phản hồi (Response):** Trả về kết quả JSON báo thành công để giao diện hiển thị thông báo và chuyển hướng người dùng.

B. Các đối tượng (Objects) sử dụng:

- **Data Transfer Object (DTO):** `schemas.TaskCreate` (Chứa dữ liệu thô từ Frontend).
- **Database Model:** `models.User` (Để cập nhật số dư), `models.Task` (Để lưu bản ghi mới).
- **Response Object:** Trả về JSON chứa `message` thành công và `task_id` mới.

2. Chức năng: Báo cáo hoàn thành kèm ảnh (Report Complete)

A. Luồng chạy của Function (Workflow):

1. **Giao diện (View):** Tại trang `my_tasks.html`, người thực hiện (Provider) nhấn "Báo hoàn thành". Một cửa sổ chọn file ảnh hiện lên.
2. **Giao tiếp (Transport):** JavaScript đóng gói file ảnh vào đối tượng `FormData` và gửi POST Request kèm `multipart/form-data` đến API.
3. **Xử lý Logic (Controller):** Hàm `report_complete` tiếp nhận file.
 - a. Sử dụng thư viện `uuid` để tạo tên file duy nhất, tránh trùng lặp.
 - b. Lưu file ảnh vật lý vào thư mục `/static/evidence/`.
4. **Cập nhật dữ liệu (Model):** Tìm bản ghi **Task** tương ứng trong Database.
 - a. Cập nhật thuộc tính `status = "waiting_confirmation"`.
 - b. Lưu đường dẫn ảnh vào thuộc tính `evidence_image`.

5. **Kết thúc:** Thực hiện `db.commit()` và trả về thông báo để giao diện cập nhật trạng thái "Chờ xác nhận".

B. Các đối tượng (Objects) sử dụng:

- **Input:** UploadFile (Đối tượng file nhị phân), `task_id` (Integer).
- **Database Model:** `models.Task` (Để lưu vết minh chứng và đổi trạng thái).

3. Chức năng: Nghiệm thu công việc (Confirm Complete)

A. Luồng chạy của Function (Workflow):

1. **Giao diện (View):** Người đăng việc (Requester) xem ảnh minh chứng, nếu đồng ý thì nhấn nút "Nghiệm thu".
2. **Điều hướng (Controller):** API `confirm-complete` nhận lệnh.
3. **Xử lý tài chính (Logic):**
 - a. Trừ số giờ đang bị khóa (`balance_locked`) của người thuê.
 - b. Cộng số giờ vào ví khả dụng (`balance_available`) của người làm.
 - c. Đổi trạng thái Task thành `completed`.
4. **Lưu nhật ký (Audit Trail):** Khởi tạo 2 đối tượng **TransactionHistory** để ghi lại lịch sử biến động số dư của cả hai bên.
5. **Hoàn tất:** `db.commit()` để chốt giao dịch vĩnh viễn trong Database.

B. Các đối tượng (Objects) sử dụng:

- **Database Models:** `models.User` (Cập nhật 2 ví), `models.Task` (Đổi status), `models.TransactionHistory` (Lưu lịch sử).

4. Chức năng: Duyệt yêu cầu nạp/rút giờ (Admin Exchange Approval)

A. Luồng chạy của Function (Workflow):

- **Giao diện (View):** Admin truy cập trang Quản trị, xem danh sách các yêu cầu nạp hoặc rút giờ đang chờ. Khi xác nhận người dùng đã chuyển khoản hoặc đủ điều kiện, Admin nhấn nút "Duyệt".
- **Điều hướng (Controller):** API `approve_deposit/withdrawal` nhận yêu cầu kèm theo `request_id` và hành động (`approve/reject`).
- **Xử lý logic (Logic):**
 - Xác định loại giao dịch: Nếu là **Nạp giờ (Deposit)**, hệ thống sẽ cộng số giờ vào `balance_available` của người dùng. Nếu là **Rút giờ (Withdraw)**, hệ thống sẽ thực hiện trừ số giờ tương ứng.
 - Cập nhật trạng thái yêu cầu trong bảng yêu cầu trao đổi thành `success`.

- **Lưu nhật ký (Audit Trail):** Khởi tạo đối tượng **TransactionHistory** để ghi lại vết giao dịch (ví dụ: "Nạp giờ qua VietQR" hoặc "Rút tiền về ngân hàng") để người dùng có thể đối soát trong Dashboard.
- **Hoàn tất:** Gọi `db.commit()` để lưu vĩnh viễn thay đổi vào hệ thống.

B. Các đối tượng (Objects) sử dụng:

- **Database Models:** `models.User` (Cập nhật số dư thực tế), `models.TransactionHistory` (Ghi lại lịch sử nạp/rút).
- **Schema:** `schemas.ExchangeRequest` (Dữ liệu đầu vào để xác định số lượng và loại hành động).

5. Chức năng: Chấp nhận nhận việc (Accept Task)

A. Luồng chạy của Function (Workflow):

- **Giao diện (View):** Tại trang "Chợ việc", người dùng xem danh sách các việc phù hợp với kỹ năng và nhấn nút "Nhận việc".
- **Điều hướng (Controller):** API `accept_task` nhận ID của Task và ID của người thực hiện (`provider_id`).
- **Xử lý logic (Logic):**
 - **Kiểm tra tính khả dụng:** Hệ thống truy vấn Database để đảm bảo Task vẫn đang ở trạng thái open và chưa bị quá hạn (Deadline).
 - **Gán nhân sự:** Cập nhật trường `provider_id` bằng ID của người vừa nhận việc.
 - **Chuyển trạng thái:** Thay đổi trạng thái Task từ open sang `in_progress` (Đang thực hiện). Tại thời điểm này, số giờ vẫn đang nằm ở "Ví khóa" của người đăng.
- **Hoàn tất:** Gọi `db.commit()` để đóng gói giao dịch. Lúc này, công việc chính thức thuộc về người nhận và không ai khác có thể nhận được nữa.

B. Các đối tượng (Objects) sử dụng:

- **Database Model:** `models.Task` (Cập nhật trạng thái và thông tin người nhận).
- **Response:** Trả về thông báo thành công để giao diện chuyển Task đó vào mục "Việc tôi nhận".